Reassembling a Dropped Trading Model

The Puzzle

A residual neural network used for trading fell apart into 97 shuffled linear layers.

Architecturally, it is a 48-block residual network:

```
x -> x + W_out(ReLU(W_inp x + b_inp)) + b_out
```

followed by one final linear readout. The shuffled pieces are:

- 48 `inp` layers (`R^48 -> R^96`)
- 48 `out` layers (`R^96 -> R^48`)
- 1 final `last` layer

Given 10,000 historical examples, recover which `inp` goes with which `out` (pairing) and in what order the 48 assembled blocks act (ordering). Together, these two assignments define the full permutation of the 97 pieces.

The search space is about `48! * 48! ~= 10^122`. The recovered answer is verified at **MSE = 3.16e-14** against the provided `pred` column on all 10,000 rows.

Main Result

The puzzle factorizes into two inverse problems:

1. **Pairing**: which `inp` and `out` pieces belong to the same residual block?
2. **Ordering**: in what sequence do those 48 recovered blocks act?

Both are independently recoverable. Multiple combinations reach the exact answer:

Pairing	Ordering
Frobenius inner product	delta-greedy residual magnitude
Multi-view fusion	graph extraction
Stagewise random-start descent	spectral-flow greedy
Pairwise tournament	Sinkhorn / insertion beam search

Every combination above reaches **MSE = 3.16e-14** after greedy full-data swap polish: try every pairwise swap of block positions, keep the swaps that reduce MSE, and repeat until no improving swap remains. No successful raw ordering method is exact by itself.

The right conclusion is not "one heuristic got lucky." Each subproblem has enough signal that several genuinely different methods solve it.

Proposition 1: Pairing Is Solved by Training-Induced Weight Correlation

Each residual block has an `inp` layer and an `out` layer that share a 96-dimensional internal space: `inp` writes into it, `out` reads from it. After shuffling, the question is which `inp` goes with which `out`. There are `48!` possible assignments.

The approach is to compute a compatibility score for every possible `(inp, out)` pair, producing a `48 x 48` score matrix, and then find the best one-to-one assignment using the Hungarian algorithm. The simplest score that works is the Frobenius inner product, which measures how strongly the two weight matrices are coordinated through the shared 96-D space:

```
score(inp_i, out_j) = |tr(W_out_j W_inp_i)|
```

Each entry in the matrix is this score for one candidate pair. Of the 2,304 entries, 48 correspond to genuine pairs and 2,256 are incorrect. On the trained network, the separation is stark: the mean score for correct pairs is **23.14x** higher than the mean for incorrect pairs, and the lowest-scoring correct pair still exceeds the highest-scoring incorrect pair. Hungarian assignment on this matrix recovers all **48/48 correct pairs**.

This coordination is created by training, not by architecture. On 10 random untrained networks with the same structure, Hungarian assignment recovers an average of **1.0/48** correct pairs — indistinguishable from chance.

To confirm this directly, a fresh network with the same architecture was trained from scratch on the same data. At several points during training, the Frobenius score matrix was recomputed and Hungarian assignment was rerun:

Epoch	Pairs correct	Mean correct score / mean incorrect score
0 (init)	2/48	1.02x
2	8/48	2.00x
5	18/48	2.64x

Epoch	Pairs correct	Mean correct score / mean incorrect score
10	23/48	2.96x
20	32/48	3.48x
60	28/48	3.20x

The pairing signal appears within the first few epochs and peaks around epoch 20. The slight decline at epoch 60 reflects overfitting — the training MSE drops to 0.0012, but the weight-space fingerprint does not monotonically strengthen. The original dropped model scores **48/48** at **23.14x**, much stronger than this 60-epoch retrain, suggesting it was trained significantly longer or with different hyperparameters.

The Frobenius inner product is not the only score that works

Five methods were tested, each measuring a different property of the `inp` / `out` interaction through the shared 96-D space. All five recover **48/48 correct pairs**.

Weight correlation — `|tr(W_out W_inp)|` : the Frobenius inner product described above. It requires no data at all — only the weight matrices. Backpropagation leaves an algebraic co-training fingerprint in the weight entries that is strong enough to perfectly identify every pair.

Operator moments — spectral invariants (trace, symmetry ratio, singular value concentration) of the composed product `W_out W_inp`. This 48x48 matrix is the linear map from input space back to input space through the block's hidden layer, ignoring the ReLU and residual connection. Correctly paired blocks produce operators with concentrated, low-rank singular value structure; incorrect pairings spread energy across many modes. Over 9,000 distinct weighting recipes of these moments recover all 48 pairs exactly — the signal is not fragile to the specific combination used.

SVD cross-alignment — geometric alignment of SVD write/read directions in the 96-D hidden space. For each `inp`, its left-singular vectors are the directions it writes into the hidden space; for each `out`, its right-singular vectors are the directions it reads from. Correctly paired layers have aligned principal modes at this hidden-space interface. Alignment in the outer 48-D input/output spaces yields only 16/48 — the co-training fingerprint lives specifically at the hidden-space interface where blocks communicate.

Affine linearization — the gated product `W_out diag(g) W_inp` where `g` is the average ReLU gate activation per hidden neuron, computed from the data. This captures the "activated" dynamics of each candidate block: not what the raw weights could do, but what they actually do under typical inputs. The full affine parameterization (operator matrix plus bias offset) is required — collapsing to scalar summaries of the bias alone destroys the signal entirely.

Multi-view fusion — weighted combination of three individually weaker signals: effective rank of `W_out W_inp`, geodesic distance between `inp` column space and `out` row space, and single-block MSE evaluated on the data. None of the three is exact on its own, but their errors are complementary. Under equal-weight fusion with robust normalization, Hungarian assignment recovers all 48 pairs — demonstrating that medium-strength signals from different domains can combine into solver-grade accuracy.

Methods that collapse or miss the shared 96-D space do not solve pairing exactly. Pure subspace geometry, `c`-only bias summaries, outer-space SVD alignment, and singular-value-only summaries of `W_out W_inp` all miss information that the exact methods keep. What matters is the coordination between the two halves of each block through their shared internal space, not a lossy summary of one side in isolation.

Proposition 2: Ordering Is Recoverable, but Mostly Through Landing Near the Answer

Once the blocks are paired correctly, the remaining problem is ordering: in what sequence do the 48 blocks act? There are `48!` possible orderings, and unlike pairing, no raw ordering method tested recovers the exact sequence directly. Every successful method lands close enough that a simple greedy polish finishes the job.

The polish step is the same everywhere: try every pairwise swap of block positions, keep swaps that reduce MSE on the full 10,000-row dataset, repeat until no improving swap remains. In the current runs it converges in 2–9 iterations.

Three genuinely different ordering methods work:

Residual-flow ordering

Delta-greedy treats the residual blocks as discrete ODE time steps and orders them by perturbation size on the data: at each step, pick the remaining block that moves the current state the least.

```
next = argmin_k ||block_k(x) - x||
```

This is the strongest raw extractor found:

- **77/97** correct raw positions, raw MSE **0.000288**
- exact after greedy polish

The intuition is dynamical: in a well-trained residual network, early blocks make small corrections and late blocks make larger ones. Delta-greedy recovers this ordering directly from the data, placing

blocks along an increasing-perturbation trajectory. It gets close enough that only a handful of swaps separate it from the exact answer.

Pairwise composition ranking

For every pair of blocks `A` and `B`, run both orderings (`A then B` vs `B then A`) on the data and compare downstream MSE. This produces a signed margin matrix:

```
margin[i,j] = MSE(j before i) - MSE(i before j)
```

Positive margin means block `i` should precede block `j`. The pairwise accuracy is **0.7757** — better than chance, but the preferences are nontransitive: `A before B` and `B before C` does not imply `A before C`. Naive sorting by win count fails (MSE **0.557**).

Three methods extract a global ordering from these nontransitive pairwise preferences:

Tournament insertion + pairwise refinement — insert blocks one at a time into the growing ordering, then improve the result by greedily swapping pairs that increase agreement with the pairwise margins. Raw MSE **0.111**, exact after polish.

Sinkhorn ranking — optimize a soft permutation matrix (doubly-stochastic via Sinkhorn iterations) to maximize expected pairwise score, with temperature annealing from multiple random restarts. Best restart raw MSE **0.123** with **9/97** correct positions, exact after polish.

Insertion beam search — greedy insertion with branching. At each step, keep the top `w` best partial orderings instead of just one. Width 5 performs best (raw MSE **0.089**, **11/97** positions), exact after polish. Wider beams (width 20) lose precision from excessive branching.

All three contain the right time direction in a noisy nontransitive form. None is exact by itself, but each lands in the correct basin.

Spectral flow

Spectral flow is the only successful ordering method that does not use prediction MSE or pairwise order comparisons. It works from linearized block Jacobians `A_k = W_out_k diag(g_k) W_inp_k`, where the average gates `g_k` are estimated from the input data, and greedily minimizes eigenvalue jitter in the cumulative product:

```
J_k = product_{t <= k} (I + A_{sigma(t)})
next = argmin_k sum((eigenvals(J_new) - eigenvals(J_prev))^2)
```

Its raw ordering is weak:

- **9/97** correct raw positions, raw MSE **0.663**

- exact after greedy polish

That is precisely why it is interesting: it gets the coarse early/late structure mostly right even though the exact order is still far from correct. Its Cayley distance from the ground truth is 41 (nearly maximal), yet its total eigenvalue jitter is **2.19** — smoother than both the ground truth (3.28) and random orderings (~3.05). It finds a different, smoother path through the same basin.

Adaptive variants fail. Several modifications were tested — using log-spectral distance, SVD of the running Jacobian, adaptive eigenvalue targets — all on the full 10,000-row dataset to remove any subsample confound. None polish to exact. The static Jacobian reads the linear blueprint cleanly; adaptive variants inject noise that pushes the ordering out of the correct basin.

Beam search over spectral flow is non-monotone. Beam widths 3, 8, 16, and 32 were tested on the full dataset with identical polish:

- width 3: fails (55/97 after polish)
- **width 8: exact** (97/97 after polish)
- **width 16: exact** (97/97 after polish)
- width 32: fails (21/97 after polish)

Widths 8 and 16 hit the basin, while 3 and 32 do not. The non-monotonicity confirms that polishability is a narrow target, not a smooth function of beam width.

Stiffness is real but not sufficient

One part of the spectral-flow claim is directly testable: does it preserve the coarse stiffness ordering? Using Frobenius norm of the linearized Jacobian as a stiffness proxy (`||A_k||_F` — higher means the block perturbs the state more), the rank correlation between block position and stiffness was measured for each ordering:

Ordering	Stiffness Spearman	Pairwise concordance	Top-12 stiff in last-12 positions
Ground truth	**0.7459**	**0.7961**	**11/12**
Spectral flow	**0.7765**	**0.8076**	**11/12**
Delta-greedy	**0.7462**	**0.7952**	**11/12**
Random (100 perms)	-0.023 ± 0.15	0.49 ± 0.05	2.8 ± 1.0

All three meaningful orderings place the stiffest blocks late — far above random. But stiffness alone is not enough: a pure Frobenius sort has perfect stiffness alignment and still gets only **15/97** raw positions with raw MSE **0.569**. Stiffness is a coarse scaffold, not a full ordering principle.

The basin is real and has a surprising shape

All five ordering methods land in the same basin of attraction: local swap polish converges to the exact ground-truth ordering from every one of them. Random permutations do not — they get stuck at MSE ~0.17 after 10 polish iterations, never approaching zero. The basin boundary is somewhere between the methods' raw orderings and random.

The surprising finding is what distinguishes basin membership. Cayley distance (minimum number of transpositions to reach ground truth) does *not* separate methods from random:

Ordering	Kendall tau to GT	Cayley to GT	Mean block displacement
Delta-greedy	5	5	0.2
Beam search	120	34	3.9
Pairwise tournament	132	42	4.2
Sinkhorn	160	38	4.8
Spectral flow	269	41	8.3
Random (5 seeds)	500–570	40–44	—

Methods and random permutations require roughly the same number of swaps to reach ground truth (Cayley ~40). Yet the methods polish to exact and random does not. What separates them is the *pattern* of displacement: methods displace a few blocks by many positions but keep the majority in roughly the right neighborhood, while random permutations displace every block uniformly. Kendall tau — which counts pairwise inversions rather than raw swap count — captures this distinction: methods range from 120 to 269, while random starts at 500.

The polish convergence paths confirm this asymmetry. All five methods reach MSE = 0 within 2–8 polish iterations, with monotonically decreasing Kendall distance at each step. Random permutations make slow progress on MSE (from ~0.7 to ~0.17 in 10 iterations) while their Kendall distance barely moves (from ~500 to ~310). The basin is not a hypersphere in swap space — it is an elongated region aligned with the coarse time direction that all five methods partially recover.

Ordering is recoverable by multiple methods, but none recover the exact sequence directly. They recover enough coarse structure — dynamical magnitude, pairwise time direction, or spectral smoothness — that greedy full-data swap polish can finish the job.

Proposition 3: The Recovered Network Has the Structure of a Trading Model

Once the blocks are put back in order, the object stops looking like 97 arbitrary layers and starts looking like a small trained trading engine. That is an interpretation, not a proof about the original training objective. But it is an interpretation backed by concrete structure.

Low-dimensional factor track

The network receives a 48-dimensional input and passes it through 48 residual blocks, each adding a correction in 48-D. Tracking the mean hidden state at each block and running PCA on the resulting trajectory shows:

- **72%** of trajectory variance in PC1
- **92%** in the first 2 PCs
- **96%** in the first 3 PCs

So the 48-block trajectory through 48-dimensional space is effectively three-dimensional.

The same compression appears at the operator level. The cumulative operator — the product of all 48 linearized block Jacobians `P = ∏(I + A_k)`, representing the composed linear effect of the full network — has effective rank that drops from about **47** at the first block to about **9** at the last. The top **5** singular values of this final cumulative operator explain about **96.5%** of its variance. The network does not use all 48 degrees of freedom equally. It compresses the state onto a narrow factor track.

Stabilizing, mostly mean-reverting dynamics

Each block's linearized Jacobian `A_k` can be decomposed into a symmetric part `S = (A+A^T)/2`, which governs contraction and expansion (diffusion), and an antisymmetric part `Ω = (A−A^T)/2`, which governs rotation. Four measurements characterize the dynamics:

All 48 blocks are locally contractive. Every block has negative Jacobian trace (range `[−6.09, −3.16]`), meaning each block shrinks volume on average. No block is expansive.

Diffusion dominates rotation. The ratio `||S||/||A|| ≈ 0.81` means 81% of each block's operator norm comes from the symmetric (contractive/expansive) part, not rotational mixing. The dynamics are primarily about pulling the state inward, not spinning it.

Most modes are mean-reverting. About **66%** of eigenvalues of the symmetric part are negative across blocks. The system is not purely contractive — some modes expand locally — but the majority are damped.

The cumulative effect is globally stable. The spectral radius of the cumulative operator `P_k` starts at about **1.05** (mildly expansive in the first few blocks) and falls to about **0.95** by the end (net

contractive). The full network does not amplify perturbations.

Together: a mostly mean-reverting, mostly dissipative system that contracts the state onto a low-rank manifold.

Phase structure is real

The recovered order is not just "small perturbations first, large last." The computation has three distinct phases:

- early blocks (positions 0–15): mean residual delta about **0.53**
- middle blocks (positions 16–31): about **0.43** — quieter than early
- late blocks (positions 32–47): about **1.10** — sharply larger
- largest single block perturbation: **3.22** at the final position

Early and late blocks also modify different features. Late blocks have **9.8x** more per-feature variance than middle blocks, and the sets of most-affected features are disjoint between early and late phases. The network processes different aspects of the input at different depths.

This phased structure is not an average-case artifact. The late/early contribution ratio was measured across six regime splits — low/high prediction magnitude, low/high input norm, low/high first principal component score — and stays between **2.00** and **2.24** in every regime. The phase structure persists and slightly sharpens in higher-signal regimes.

Shock handling is highly structured

The cleanest causal evidence comes from perturbation analysis. Instead of summarizing states, this test injects calibrated shocks along different input directions — top input PCs (the directions of greatest variance), bottom input PCs (the directions of least variance), and individual target-correlated features — then tracks the response norm through all 48 blocks.

The answer is highly non-generic:

- top-PC factor shocks are **amplified and carried through depth**, peaking at the final blocks
- bottom-PC nuisance shocks are **damped immediately**, peaking at the input and decaying

Quantitatively:

- top-PC shocks: damping ratio **1.8756** (amplifying), prediction shift **0.0305**
- bottom-PC shocks: damping ratio **0.6937** (damping), prediction shift **0.00138**

The prediction-shift gap is **22x**: the network's final output is 22 times more sensitive to perturbations along high-variance directions than along low-variance directions.

This separation was retested inside every regime split (low/high prediction magnitude, low/high input norm, low/high PC1 score). The top/bottom prediction-shift ratio stays between **15.4x** and **24.1x** in

every regime. Top-PC shocks remain amplifying (damping ratio 1.35–2.26) and bottom-PC shocks remain damped (damping ratio 0.52–0.82) everywhere.

So the network is not just "contractive." It selectively preserves factor-like directions and suppresses nuisance directions, and it does so consistently across the input distribution.

The best compact surrogate is observer-like

The trajectory through the 48 blocks can be projected into a low-rank PCA latent space. The question is: what kind of dynamical system best describes the block-to-block updates in that latent space?

Two families of models were fit at ranks 3, 5, 7, and 9:

Autonomous models predict the next latent state from the current one alone: `z_{t+1} = a + z_t F`. Within this family, at ranks 3–7 the best fit is **quadratic energy descent** — a model where only the symmetric (dissipative) part of the drift matrix is kept and projected to be negative semidefinite, enforcing norm decay at every step like a gradient flow on a quadratic Lyapunov function. At rank 9, **OU + rotation** fits best — the symmetric part is kept stable but the antisymmetric (rotational) part is preserved, allowing the latent state to spiral inward rather than descend straight down.

Observer models add a correction term from the full high-dimensional state: `z_{t+1} = a + z_t F + r_t C`, where `r_t` is the reconstruction residual — the difference between the actual 48-D state and its low-rank projection. This lets the surrogate correct its latent estimate using information that the low-rank projection missed.

The observer does **not** win on every metric. Raw teacher-forced trajectory MSE stays similar and is sometimes slightly worse. But it wins on the metrics that matter for interpretation — final prediction accuracy and shock-response fidelity:

- rank 9 prediction MSE: **0.8466 → 0.8015** (observer)
- average shock damping gap: **1.1530 → 0.6582** (observer)
- average shock prediction-shift gap: **0.0130 → 0.0083** (observer)

The observer better reproduces how the real network handles perturbations, which is the behavioral signature that distinguishes this network from a generic contractive map.

So the best compact summary is not "a low-rank autonomous ODE." It is closer to:

a low-rank stable factor system with observer-like correction from the current high-dimensional state.

Without the puzzle wrapper, this model reads as a factor-compressing residual forecaster: stabilize noisy state, move it onto a narrow factor track, then apply sharper late corrections before a final linear prediction.

Conclusion

Across the experiments in this repo, all 25 pairing × ordering combinations reconstruct the same exact 97-piece permutation, and random-start polish stalls around MSE 0.17–0.19 instead of reaching zero. The end-to-end evidence is robustness across independent methods, not one lucky path.

Two boundaries are worth stating. The uniqueness evidence is local, not a global proof — we have not exhaustively checked all `10^122` alternatives. And the trading-model interpretation in Proposition 3 is structural, not a recovered training objective — the network's dynamics are consistent with a factor-filtering forecaster, but we cannot prove that was the intent behind its training.

Generalization: beyond this puzzle

The key technique behind the pairing solution — the Frobenius inner product `|tr(W_A^T W_B)|` as a co-training fingerprint — relies on a specific mechanism: shared gradient flow through bilinear couplings during training creates coordinated weight structure that persists in the final model. If this mechanism is general, the same technique should work on other architectures wherever two weight matrices interact through a dot product in the forward pass.

Preliminary experiments on two billion-parameter language models confirm this:

	Puzzle (48-block ResNet)	**Qwen3-1.7B (28L transformer)**	**Qwen3.5-2B (24L hybrid)**
Bilinear coupling	W_out · W_inp	Q · K	Q · K, alpha · beta
Fingerprint SNR	23x	13x	83x / 5.5x
Shuffle & recover	48/48 (100%)	24/28 (86%)	6/6, 18/18 (100%)
Phase structure (late/mid)	2.6x	10.78x	2.19x

The Frobenius fingerprint recovers correct Q-K layer pairings from shuffled weights on a standard 28-layer transformer (86% accuracy, chance = 3.6%). On the hybrid Qwen3.5-2B — which alternates Gated DeltaNet (linear attention) with standard softmax attention — two distinct fingerprints appear: the Q-K coupling (83x SNR, 6/6 perfect) and a novel DeltaNet alpha-beta coupling (5.5x SNR, 18/18 perfect). The alpha-beta fingerprint was not predicted in advance; it emerged because the DeltaNet state update rule creates a bilinear interaction between these projections, analogous to the Q-K dot product.

Phase structure also generalizes: late layers make larger corrections than middle layers across all three architectures.

Not everything transfers. Effective rank collapse (47 → 9 in the puzzle) does not appear in LLMs, which maintain near-full rank because they serve diverse tasks. MLP projection pairs (gate-down, up-down) show no fingerprint — the elementwise multiplication coupling is too diffuse.

The core prediction: **any architecture with bilinear projection couplings in the forward pass will exhibit recoverable co-training fingerprints in the weight matrices.** The coupling tightness predicts the fingerprint strength. A systematic study across model families, scales, and training regimes is left for future work.

The Answer

```
43 34 65 22 69 89 28 12 27 76 81 8 5 21 62 79 64 70 94 96
4 17 48 9 23 46 14 33 95 26 50 66 1 40 15 67 41 92 16 83
77 32 10 20 3 53 45 19 87 71 88 54 39 38 18 25 56 30 91 29
44 82 35 24 61 80 86 57 31 36 13 7 59 52 68 47 84 63 74 90
0 75 73 11 37 6 58 78 42 55 49 72 2 51 60 93 85
```

Verified at **MSE = 3.16e-14** on all 10,000 rows.